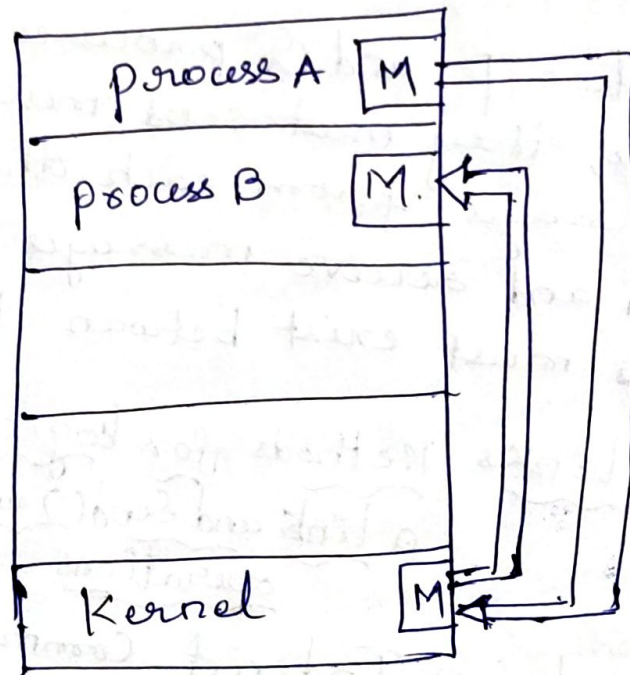


## ⇒ Message Passing IPC:-

↳ It is mechanism that allows the processes to communicate and to synchronize their actions without sharing the same address space.

↳ It is particularly useful in a distributed environment.

↳ In this communicating processes may reside on different computers connected by a network.



eg: Internet Chat: you are sending message from your (process) system and your friend receiving message in his system located at remote place.

↳ It provides atleast two operations

- 1) Send (message)
- 2) Receive (message).



⇒ Messages sent by a process can be of either fixed (or) variable size. (Here it is length of message) either small (or) Big.

Fixed Size:- The system-level implementation is straight forward.

But makes the task of programming more difficult.

Variable Size:- Requires a more complex system-level implementation. But the programming task becomes simpler.

Lets say: Two P and Q processes want to communicate, they must send messages to and receive messages from each other.

→ To send and receive messages, a communication link must exist between them.

⇒ Types of Links / Methods for logically implementing a link and send() and receive() operations.

- \* Direct (or) Indirect Communication
- \* Synchronous (or) Asynchronous Communication
- \* Automatic (or) Explicit buffering.

There are several issues related with features

Like: → \* Naming

→ \* Synchronization

→ \* Buffering while implementing the links.



➔ Direct Communication:- Each process that wants to communicate must explicitly mention the name of recipient (or) sender of communication.

\*  $\text{Send}(P, \text{message})$  - Send a message to process  $P$ .  
\*  $\text{Receive}(Q, \text{message})$  - Receive a message from process  $Q$ .

- \* A link is associated with exactly two processes.
- \* This exhibits symmetry in addressing, that means both the sender process and the receiver process must name each other to communicate.

\* Another Variation in direct Communication:

Here sender names the recipient; the recipient is not required to name the sender.

\*  $\text{Send}(P, \text{message})$  - Send a message to process  $P$ .  
\*  $\text{Receive}(id, \text{message})$  - Receive a message from any process. The variable  $id$

Asymmetry  
in addressing.

is set to the name of the process with which communication has taken place.

Disadvantages:- (In both symmetry and asymmetry)

- \* Limited Modularity of the resulting process definitions.
- \* Changing the identifier of a process may necessitate examining all other process definitions.



## ⇒ Indirect Communication

The messages are sent to and received from mailboxes (or) ports.

\* A mailbox is an object into which messages can be placed by processes and from which messages can be removed.

\* Each mailbox has a Unique Identification.

\* Two processes can communicate only if the processes have a shared mailbox.

\* Send (A, message) - Send a message to mailbox A.

\* Receive (A, message) - Receive a message from mailbox A.

\* A link may be associated with more than two processes.

\* A link is established only when processes have a shared mailbox.

\* A mailbox may be owned either by a process. (or) by the operating system.



# ⇒ Synchronous (Blocking) (or) Asynchronous (non blocking)

- \* Blocking send :- The sending process is blocked until the message is received by the receiving process (or) by the mailbox.
- \* Non blocking send :- The sending process sends the message and resumes operation.
- \* Blocking receive :- The receiver blocks until a message is available.
- \* Non blocking receive :- The Receiver retrieves either a valid message (or) a null.

## ⇒ Buffering:

↳ It is a temporary Queue, where messages resides, and the messages are exchanged by ~~processes~~ communicating processes.

↳ These are implemented in three ways

- \* Zero capacity :- The Queue has maximum length of zero, which means only one message can be passed at a time. Sender must block until the recipient receives the message.

- \* Bounded Capacity :- The Queue has finite length "n". Thus at most n messages can reside in it.

- \* Unbounded Capacity :- The Queue length is potentially infinite, thus any number of messages can wait in it. Sender never blocks.



## ⇒ Types of Message passing Mechanisms.

- \* Files
- \* pipes
- \* FIFO's

How pipe works:  
1. When you write data to fd[1], kernel stores it in internal memory buffer.  
2. When you read data from fd[0], the kernel retrieves data from this buffer.

\* Pipes:- (unnamed pipe) H) Once all data is read buffer becomes empty



\* pipes are generally used in Communication between related processes.  
for example: Parent and child process Communication  
It will have two ends

\* pipe provides Unidirectional Communication i.e either read or write.

\* System call `pipe()` function is used to Create a unidirectional pipe for IPC.

\* It needs `#include <unistd.h>`

Syntax:- `int pipe(int pipefd[2]);`

0 ← on Success: It returns two file descriptors  
`pipefd[0]` → reading end  
`pipefd[1]` → writing end.

\* process which will receive the data should use `pipefd[0]`.

\* process ~~that~~ that wants to send the data should use `pipefd[1]`.

\* Since communication is b/w related processes use `fork()` to create processes.

on fail  
yep



pipe program:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
```

```
int main()
```

```
{ int fd[2], n;
```

```
  char buffer[100];
```

```
  pid_t pid;
```

```
  pipe(fd); // creates unidirectional pipe with two
             ends fd[0] and fd[1].
```

```
  pid = fork();
```

```
  if (pid > 0) // parent
```

```
  { printf("parent passing the message to child\n");
    write(fd[1], "welcome\n", 8); // fd[1] is write end
                                   of pipe.
```

```
  }
```

```
  else // child
```

```
  { printf("child received the message\n");
```

```
    n = read(fd[0], buffer, 100); // fd[0] is read end of
                                   the pipe.
```

```
    write(1, buffer, n);
```

```
  }
```

```
}
```

Note: 1) Data loss on process termination.

(in pipe)

2) To check pipe descriptors use

ls -l /proc/<pid>/fd/ after creating pipe.

Note: ① file descriptors do not point to actual files on file system (or) disk file.

→ File descriptors refers to a special in-memory buffer maintained by kernel in memory.

→ file descriptors are given a number by OS.



→ can be used as ~~file~~ Normal files.

## \* Named pipes:- FIFO

→ This method can be used for Communication between different process (or) same processes

→ It is Bi-directional.

→ No need of parent-child relationship; Can be between any processes

→ Exists even after process terminate. (until deleted).

Header files:- #include <sys/types.h>  
#include <sys/stat.h>

Syntax:- System call to create Named pipe.  
is mkfifo ( )

→ int mkfifo (Const char \*pathname, mode\_t mode);

on Success: Returns 0.

on error: Returns -1.

↓  
Name of the file

↓  
Permissions  
read.  
write.  
Execute

It consists of three programs:

program<sub>1</sub>: To create a named pipe

program<sub>2</sub>: process that will write into the pipe (Sender process)

program<sub>3</sub>: process that will receive the data from pipe (Receiver process).

→ FIFO named pipe works in Blocked Mode by default.

→ Both reader and writer process must be present at the same time

eg: Teacher student.  
Sender receiver.



1) program: Creating fifo | named pipe.

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
```

```
int main()
```

```
{ int res;
```

```
res = mkfifo("fifo", 0777);
```

```
printf("Named pipe Created\n");
```

```
}
```

ls -l fifo.

o/p: prwxr-xr-

↓  
pipe

o/p: Named pipe  
Created

2) program: Sender process to write data into pipe  
(producer process)

gcc -o filename filename.c

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <fcntl.h>
```

```
int main()
```

```
{ int fd;
```

```
fd = open("fifo", O_WRONLY);
```

```
write(fd, "Hello world", 11);
```

```
printf("Sender process having PID %d sent the data\n",  
getpid());
```

```
}
```

3) program: Receiver process to receive data from pipe.

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <fcntl.h>
```

```
int main()
```

```
{ char buffer[100];
```

```
int fd;
```

```
fd = open("fifo", O_RDONLY);
```

```
n = read(fd, buffer, 100);
```

```
write(1, buffer, n);
```

```
printf("Receiver process received data having  
pid %d\n", getpid());
```

```
}
```



// Program 3: Receiver process to read the data from pipe.

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
int main()
```

```
{
    char buffer[100];
```

```
    int fd;
```

```
    fd = open("fifo", O_RDONLY);
```

```
    read(fd, buffer, 100);
```

```
    printf("Reader process started\n");
```

```
    printf("Data received by receiver %d is: %s\n",
           getpid(), buffer);
```

O/p:- After running program 2 and program 3 simultaneously.

\$ ./sender & ./receiver

Sender process having PID - sent the data

Reader process started

Data received by receiver - is: Hello world.

Note:- To run in Non-Block (unblock) mode in program 2 a simple modification is

```
fd = open("fifo", O_NONBLOCK, O_WRONLY)
```

Similarly in program 3

```
fd = open("fifo", O_NONBLOCK, O_RDONLY)
```





## Process Synchronization:-

process synchronization:- It is a technique used to control the execution of multiple processes to ensure correct execution.

\* when multiple process often need to access shared resources, they must be synchronized to prevent conflicts.

\* Synchronization is a technique to control the execution of multiple processes (or) threads to ensure correct execution when accessing shared resources.

\* It prevents problems like race conditions, data inconsistency and deadlocks.

### Types of Synchronization:-

1. process synchronization
2. Thread synchronization

process synchronization: It is a technique used in operating system, to ensure that multiple process can run simultaneously without interfering with each other while access shared resources.

### Why process synchronization:-

- \* prevents data inconsistency
- \* Avoids race condition
- \* Ensure fairness
- \* Maintains system stability



## ⇒ Critical Section problem:

A portion of a program where a process accesses shared resources like Variables, files, memory, databases.

To ensure proper Synchronization, a process must follow some rules.

1. Mutual Exclusion: Only one process should enter the critical section at a time
2. progress: If no process is in the critical section, other process should be able to enter.
3. Bounded Waiting: A process should not wait indefinitely to enter the critical section